

Operating Systems Laboratory Report

Course: Operating Systems / Operating Systems Laboratory

Institution: Indian Institute of Technology Bombay

Instructor / Course Material: Prof. Mythili Vutukuru

Repository: `sunny4095/OS-Labs`

Scope: Programming laboratories only; lab quizzes excluded

1. Introduction

This report summarizes the programming exercises completed as part of the Operating Systems laboratory work based on the IIT Bombay Operating Systems course material. The laboratory exercises provide practical experience with the mechanisms and abstractions studied in operating systems, progressing from basic Linux process inspection and system programming to process management, inter-process communication, memory management, synchronization, multithreading, and modifications to the xv6 teaching operating system.

The official IIT Bombay Operating Systems material is structured around processes, memory management, concurrency, I/O and filesystems, and xv6-based operating-system implementation. The programming exercises are intended to complement the theoretical material by requiring direct interaction with Linux system calls and modification of an operating-system kernel.

The submitted work is maintained in a GitHub repository containing the laboratory implementations and supporting xv6 source trees. The repository also contains separate directories for lab quizzes; these have deliberately been excluded from this report.

2. Objectives of the Laboratory Work

The overall objectives of the laboratory exercises were:

- To become comfortable with Linux command-line tools and system interfaces.
- To understand the lifecycle and representation of processes.
- To work directly with system calls such as `fork`, `exec`, `wait`, and related process-management primitives.
- To implement and understand a shell that creates and manages processes.
- To study different mechanisms for inter-process communication.
- To understand virtual memory and page-management mechanisms through xv6.
- To implement memory-management optimizations such as Copy-on-Write.
- To understand concurrency using POSIX threads.
- To implement synchronization mechanisms and reason about race conditions.
- To understand how user-level threads interact with kernel scheduling and context switching.
- To gain experience reading and modifying an existing operating-system kernel rather than treating the OS as a black box.

The official course material explicitly emphasizes processes, virtual memory, concurrency, filesystems, and xv6 as the major practical components of the course.

3. Laboratory Environment

The experiments primarily use a Linux development environment and the C programming language.

The major tools and technologies involved were:

- Linux / GNU utilities
- C programming
- POSIX system calls
- POSIX threads (`pthread`)
- GCC
- GNU Make
- Git
- QEMU
- xv6
- Process and memory inspection through `/proc`
- Pipes and other IPC mechanisms
- Synchronization primitives
- x86-oriented xv6 kernel code

For the xv6 laboratories, the supplied xv6 source code was compiled and executed through the QEMU-based environment. This provided a controlled environment in which kernel modifications could be tested without affecting the host operating system.

4. Lab 1 — Introduction to Linux Tools

4.1 Objective

The first laboratory introduced the Linux environment and provided practical experience with process inspection, system information, the `/proc` filesystem, and process monitoring.

The official exercise begins with Linux commands for examining processes and then moves toward inspecting information exposed by the Linux kernel through `/proc`.

4.2 Work Completed

The submitted `lab1` directory contains the code associated with the introductory Linux exercises.

The exercises involved:

- Listing currently running processes.
- Examining different output formats of `ps`.
- Constructing process trees.

- Inspecting process identifiers such as PID and PPID.
- Examining process state and command information.
- Inspecting CPU and system information.
- Reading information from `/proc`.
- Using commands such as `lscpu`, `uname`, `uptime`, `ps`, and `top`.
- Observing the resource consumption of long-running processes.

A central part of the lab was understanding that information displayed by utilities such as `ps` and `top` is ultimately obtained from kernel-maintained process information.

4.3 Concepts Learned

This laboratory provided practical understanding of:

- Process identifiers.
- Parent-child process relationships.
- Process states.
- Linux process accounting.
- The `/proc` pseudo-filesystem.
- CPU and memory utilization.
- Basic Linux system administration tools.

4.4 Outcome

The laboratory established the foundation required for later experiments involving process creation, scheduling, memory management, and synchronization.

5. Labs 2–3 — Process Management and Shell Programming

The repository stores the material associated with these laboratories together in `lab2_lab3`. It contains warm-up material, shell code, and a solution directory containing the completed shell implementation.

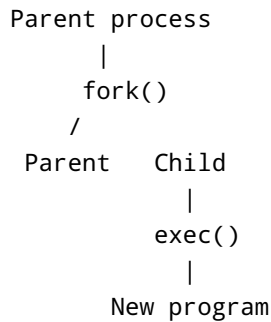
5.1 Objective

The purpose of these exercises was to understand how a Linux shell executes commands and how processes are created, executed, synchronized, and terminated.

The official shell exercise specifically introduces `fork`, `exec`, `exit`, and `wait`, and asks students to build a simple shell similar to Bash.

5.2 Process Creation

The exercises introduced the standard process lifecycle:



The child process can replace its program image using one of the `exec` variants, while the parent can use `wait` to synchronize with its child.

This demonstrates an important UNIX design principle: **process creation and program execution are separate operations.**

5.3 Shell Implementation

The repository contains `my_shell.c`, indicating the implementation of a custom command shell.

The shell performs the following general sequence:

1. Read a command from standard input.
2. Parse the command and its arguments.
3. Create a child process.
4. Execute the requested program in the child.
5. Wait for the child when executing in the foreground.
6. Continue accepting commands.

This required direct use of process-management system calls rather than relying on a higher-level process abstraction.

5.4 Foreground and Background Execution

An important aspect of shell programming is distinguishing between:

- **Foreground execution**, where the shell waits for the command to terminate.
- **Background execution**, where the shell immediately returns to the command prompt while the child continues executing.

This exercise demonstrated how process synchronization affects the observable behavior of a command interpreter.

5.5 Concepts Learned

The laboratory provided practical understanding of:

- `fork()`
- `exec()` and its variants

- `wait()`
- Process creation
- Parent-child relationships
- Process termination
- Foreground/background execution
- Command parsing
- Shell architecture

5.6 Outcome

The laboratory transformed the abstract process model into an executable system. Instead of simply studying process creation theoretically, the implementation demonstrated how a real shell uses operating-system primitives to launch programs.

6. Lab 4 — xv6 Process Management

The `lab4` submission contains an xv6 source tree along with the original xv6 source used as a reference.

6.1 Objective

This laboratory moved from Linux user-space programming into operating-system implementation.

The objective was to understand how process management is implemented inside xv6, including the kernel structures and mechanisms responsible for creating, scheduling, switching, and terminating processes.

6.2 xv6 Process Representation

A process in xv6 is represented using a kernel-level process structure containing information such as:

- Process state.
- Process identifier.
- Parent process.
- Address-space information.
- Kernel stack.
- Trap frame.
- Context used for context switching.
- Open files and other process resources.

Understanding this structure is important because process management is no longer being observed through utilities such as `ps`; instead, the implementation itself is being inspected.

6.3 Process Lifecycle

The laboratory required understanding the transition between process states, including states such as:

UNUSED → EMBRYO → RUNNABLE → RUNNING



SLEEPING



RUNNABLE

RUNNING → ZOMBIE

The kernel is responsible for maintaining these transitions and ensuring that resources are properly handled when processes terminate.

6.4 Context Switching

A major concept in xv6 is the separation between:

- A process's user-level execution state.
- Its kernel execution state.
- The saved CPU context required to resume execution.

Context switching allows the CPU to move between processes while preserving enough state to resume each process later.

6.5 Key Concepts

The laboratory strengthened understanding of:

- Process Control Blocks.
- Process states.
- Kernel stacks.
- Trap frames.
- Context switching.
- Scheduler interaction.
- `fork` and process creation.
- Process termination.
- Kernel/user transitions.

6.6 Outcome

This laboratory provided a transition from using operating-system abstractions to understanding their implementation. xv6 served as a compact but realistic example of how a UNIX-like kernel manages processes.

7. Lab 5 — Memory Management in xv6

The repository contains a separate `lab5` directory with an xv6 source tree.

7.1 Objective

The objective of this laboratory was to understand how xv6 manages virtual memory and translates virtual addresses into physical memory.

The laboratory required reasoning about page tables, virtual addresses, physical pages, address spaces, and the interaction between processes and the memory-management subsystem.

7.2 Virtual Address Spaces

Each process operates within its own virtual address space.

A virtual address is translated through the process's page table:

```
Virtual Address
  |
  v
Page Table
  |
  v
Physical Address
```

This abstraction allows processes to use independent address spaces while the kernel controls how virtual pages map to physical memory.

7.3 Page Tables

The xv6 memory-management code maintains page-table entries that contain information about:

- Physical page mappings.
- Validity.
- Permissions.
- User/kernel accessibility.
- Read/write permissions.

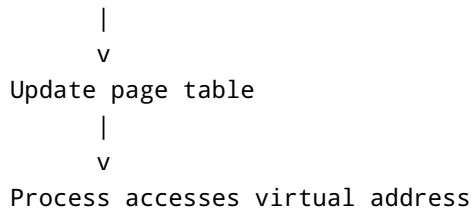
Understanding these entries is essential for implementing memory-management features.

7.4 Memory Allocation and Deallocation

The laboratory involved understanding how physical pages are allocated and returned to the system.

A typical page-management sequence is:

```
Request memory
  |
  v
Allocate physical page
```



When memory is no longer required, the corresponding physical pages must be released and their mappings removed.

7.5 Page Faults

Memory-management mechanisms depend on the CPU detecting invalid or disallowed accesses.

A page fault transfers control to the kernel, which can determine whether the access is valid and what action should be taken.

This concept becomes particularly important in the later Copy-on-Write work.

7.6 Outcome

The laboratory provided a detailed understanding of the relationship between:

- Virtual memory.
- Physical memory.
- Page tables.
- Page-table entries.
- Address translation.
- Memory allocation.
- Kernel memory-management code.

8. Lab 6 — Inter-Process Communication

The repository contains a dedicated `lab6-IPC` directory with separate implementations for multiple parts, socket code, and warm-up exercises.

8.1 Objective

The goal of this laboratory was to study mechanisms that allow independent processes to exchange information.

The repository divides the implementation into multiple parts, including producer/consumer-style code and socket-based communication.

8.2 IPC Motivation

Processes normally have isolated address spaces. Consequently, sharing information between them requires explicit operating-system mechanisms.

IPC mechanisms provide controlled ways for processes to communicate while maintaining process isolation.

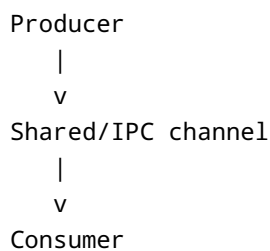
8.3 Producer-Consumer Communication

The repository contains:

- `producer_sol.c`
- `consumer_sol.c`

for one of the IPC exercises.

The producer-consumer model consists of:



The producer generates data while the consumer receives and processes it.

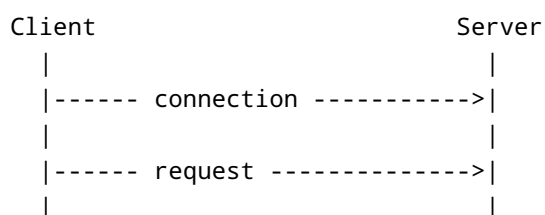
This model introduces important issues such as synchronization, blocking, ordering, and buffer management.

8.4 Socket-Based Communication

The laboratory also contains a `sockets` component, indicating practical work with network-style IPC.

Sockets provide a more general communication abstraction that can be used both locally and across machines.

The typical architecture is:



```
|<----- response -----|
|                               |
```

8.5 Concepts Learned

The laboratory provided practical exposure to:

- Process isolation.
- IPC.
- Producer-consumer communication.
- Pipes or related communication mechanisms.
- Blocking and synchronization.
- Socket programming.
- Client-server communication.

8.6 Outcome

The laboratory demonstrated that process isolation does not prevent cooperation. Instead, the operating system provides explicit communication mechanisms through which processes can safely exchange information.

9. Lab 7 — Copy-on-Write and xv6 Memory Optimization

The `lab7` directory contains dedicated implementations related to Copy-on-Write, including `copy_on_write_fork` and `cow`, together with xv6 source trees.

9.1 Objective

The purpose of this laboratory was to implement and understand **Copy-on-Write (CoW)** memory management.

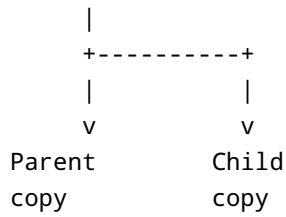
The conventional `fork()` implementation copies the parent's user-space memory into the child. This can be expensive because a large amount of memory may be copied even when the child immediately replaces its address space using `exec()`.

Copy-on-Write avoids unnecessary copying.

9.2 Conventional Fork

Without CoW:

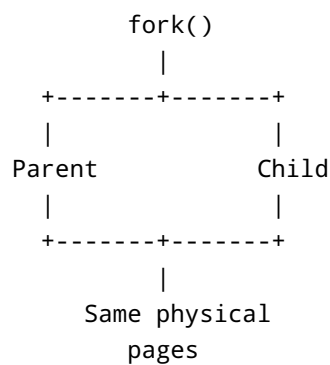
```
Parent memory
|
| fork()
```



Every page is duplicated immediately.

9.3 Copy-on-Write Fork

With CoW:



The parent and child initially share physical pages.

The pages are marked read-only. When one process attempts to modify a shared page, a page fault occurs.

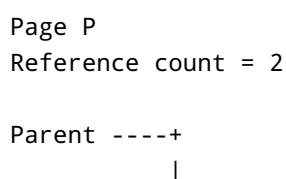
The kernel then:

1. Allocates a new physical page.
2. Copies the original page.
3. Updates the faulting process's page table.
4. Makes the new page writable.
5. Updates the reference count of the original page.

9.4 Reference Counting

Copy-on-Write requires the kernel to know how many processes currently reference each physical page.

Conceptually:



```
      +-----> Page P
      |
Child  -----+
```

After the child writes:

```
Parent  ----> Page P
Child  -----> Page Q
```

The reference count of **P** is reduced, while **Q** receives its own reference.

9.5 Page Fault Handling

The implementation therefore connects several OS concepts:

- Page tables.
- Page permissions.
- CPU exceptions.
- Page faults.
- Physical page allocation.
- Reference counting.
- Process address spaces.

The official IIT Bombay material also uses Copy-on-Write as an important example of modern memory-management behavior.

9.6 Outcome

This laboratory demonstrated how an optimization that appears simple at the conceptual level requires coordination between the process subsystem, page allocator, page tables, exception handling, and synchronization.

10. Lab 8 — Pthreads and Synchronization

The **lab8** directory contains three parts together with a dedicated **pthread** directory and warm-up material.

The submitted work includes a master-worker implementation, reader-writer locks, and a semaphore-like synchronization mechanism.

10.1 Part A — Master-Worker Programming

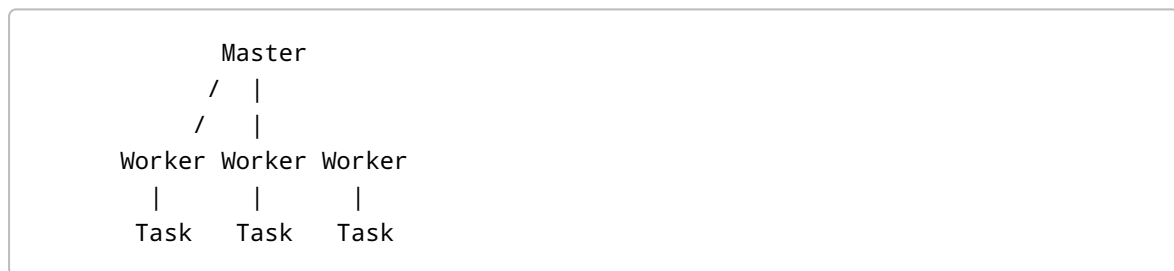
The repository contains:

- `master-worker.c`
- `master-worker-bull.c`

along with compiled objects.

The master-worker architecture divides work between a coordinator and multiple worker threads.

Conceptually:



This model demonstrates how a large task can be divided into independent pieces and processed concurrently.

Important concerns include:

- Thread creation.
- Work distribution.
- Shared data.
- Synchronization.
- Thread termination.
- Correctness under concurrent execution.

10.2 Part B — Reader-Writer Locks

Part B implements reader-writer locks in both reader-preference and writer-preference forms. The repository contains:

- `rwlock-reader-pref.cpp`
- `rwlock-writer-pref.cpp`
- `rwlock.h`
- Corresponding test programs.

A reader-writer lock allows:

- Multiple readers simultaneously.
- Only one writer at a time.
- No reader while a writer is modifying the shared resource.

Reader Preference

Reader preference allows readers to enter as long as no writer currently holds the lock.

This can maximize read throughput but can potentially starve writers if readers continuously arrive.

Writer Preference

Writer preference gives waiting writers priority over newly arriving readers.

This reduces writer starvation but may reduce read concurrency.

The exercise demonstrates an important principle in synchronization:

A synchronization mechanism must not only guarantee mutual exclusion; its scheduling policy also affects fairness and performance.

10.3 Part C — Zemaphore

Part C contains:

- `zemaphore.c`
- `zemaphore.h`
- `test-zem.c`
- `test-toggle.c`
- A test script.

The exercise implements a semaphore-like synchronization primitive using lower-level synchronization facilities.

The abstraction maintains a counter representing the number of available permits.

Conceptually:

```
wait()
|
+-- counter > 0 --> decrement and continue
|
+-- counter == 0 -> block

post()
|
+-- increment counter
|
+-- wake a waiting thread
```

This demonstrates how higher-level synchronization primitives can be constructed from more fundamental locking and condition-waiting mechanisms.

10.4 Concepts Learned

This laboratory covered:

- POSIX threads.
- Thread creation and joining.
- Shared-memory concurrency.
- Race conditions.
- Mutual exclusion.
- Reader-writer synchronization.
- Starvation.
- Fairness.
- Semaphores.
- Thread-safe programming.

10.5 Outcome

The laboratory provided practical experience with one of the most difficult aspects of systems programming: ensuring correctness when multiple execution flows access shared state concurrently.

11. Lab 10 — Threads in xv6

The repository's `lab10/sol1` directory contains an `xv6-threads` implementation and an accompanying archive.

11.1 Objective

The goal of this laboratory was to understand how threads can be implemented inside an operating system and how they differ from independent processes.

Unlike a process, threads belonging to the same process can share resources such as:

- Address space.
- Code.
- Global data.
- Open files.

However, each thread requires its own execution context, including its own stack and CPU register state.

11.2 Process vs Thread

A simplified comparison is:

Feature	Process	Thread
Address space	Separate	Shared
Code	Separate address space	Shared

Feature	Process	Thread
Global memory	Separate	Shared
Stack	Own	Own
CPU context	Own	Own
Communication	IPC required	Shared memory
Creation overhead	Higher	Lower

11.3 Thread Context

A thread context contains the state necessary to suspend and resume its execution.

During a context switch:

```

Thread A
  |
  | save context
  |
  v
Scheduler
  |
  | load context
  |
  v
Thread B

```

The kernel therefore needs mechanisms to:

1. Create a thread.
2. Allocate its execution stack.
3. Initialize its context.
4. Schedule it.
5. Switch between threads.
6. Terminate and clean up the thread.

11.4 Shared Address Space

One of the main distinctions from processes is that threads within a process share the same virtual address space.

This makes communication efficient but also introduces race conditions.

For example:


```
counter++;
```

is not necessarily atomic when multiple threads execute it simultaneously because it may involve:

```
read counter  
modify value  
write counter
```

Two threads can interleave these operations and lose updates.

Thus thread implementation naturally connects back to the synchronization concepts studied in the previous laboratory.

11.5 Outcome

This laboratory combined the earlier concepts of:

- Processes.
- Context switching.
- Scheduling.
- Address spaces.
- Concurrency.
- Synchronization.

It demonstrated how an operating system can support multiple execution flows while allowing them to efficiently share resources.

12. Integration of Concepts Across the Laboratories

The laboratories were not isolated programming exercises. They progressively built an understanding of the operating system as a complete system.

The progression can be summarized as:

```
Linux tools  
  |  
  v  
Processes  
  |  
  v  
fork / exec / wait  
  |  
  v
```

```
Shell
  |
  v
IPC
  |
  v
xv6 process management
  |
  v
Virtual memory
  |
  v
Copy-on-Write
  |
  v
Threads
  |
  v
Synchronization
  |
  v
Threads inside xv6
```

Each stage builds upon mechanisms introduced earlier.

For example, understanding `fork()` is necessary to understand Copy-on-Write. Understanding virtual memory is necessary to understand why CoW can be implemented by modifying page-table permissions. Similarly, understanding process context switching helps explain how kernel-level threads are scheduled.

13. Major Technical Concepts Learned

13.1 Processes

A process is an executing program together with its execution state and operating-system resources.

The laboratories demonstrated both the user-space interface to processes and the kernel implementation of processes.

13.2 System Calls

System calls provide the interface between user programs and the operating-system kernel.

Important calls encountered include:

- `fork`

- `exec`
- `wait`
- `exit`
- Memory-management system calls
- IPC-related system calls

These calls demonstrate the boundary between user mode and kernel mode.

13.3 Context Switching

Context switching allows the processor to stop one execution flow and resume another.

The saved context contains enough CPU state to continue execution correctly.

This mechanism is fundamental to both process scheduling and thread scheduling.

13.4 Virtual Memory

Virtual memory provides each process with an independent address space.

Page tables translate virtual addresses into physical addresses and enforce memory-access permissions.

13.5 Copy-on-Write

Copy-on-Write improves the efficiency of `fork()` by postponing memory copying until a shared page is actually modified.

This illustrates how virtual-memory mechanisms can be used to optimize process creation.

13.6 Concurrency

Concurrency occurs when multiple execution flows make progress during overlapping periods.

The laboratories demonstrated that concurrency introduces problems that do not occur in sequential programs, especially when threads share mutable data.

13.7 Synchronization

Synchronization mechanisms coordinate concurrent execution and prevent incorrect interleavings.

The laboratory work included:

- Locks.
- Reader-writer locks.
- Semaphores.
- Waiting and signaling.
- Fairness considerations.

13.8 IPC

IPC allows isolated processes to communicate.

The exercises demonstrated both local process communication and socket-based communication.

14. Tools and Technologies

The major technologies used throughout the laboratory work were:

Technology	Usage
C	Systems programming and kernel modifications
Linux	Development and experimentation environment
GCC	Compilation
Make	Build automation
Git	Source-code management
QEMU	xv6 execution and testing
xv6	Teaching operating-system kernel
<code>/proc</code>	Linux process/system inspection
POSIX APIs	Process, thread and synchronization programming
Sockets	Communication between processes

15. Challenges Encountered

The laboratory exercises presented several progressively difficult systems-programming challenges.

15.1 Understanding Existing Kernel Code

Unlike ordinary programming assignments, xv6 required modifying an existing codebase with multiple interacting subsystems. Understanding the control flow between system calls, traps, memory management, and scheduling was therefore essential.

15.2 Debugging Process Behavior

Process execution is inherently nondeterministic. The relative ordering of parent and child execution can vary depending on scheduling, making synchronization using `wait` and related mechanisms important.

15.3 Memory Management

Memory-management bugs can manifest far away from the code that introduced them. Page-table mappings, permissions, physical-page allocation, and reference counts must remain consistent.

15.4 Concurrent Programming

Threaded programs introduced race conditions and synchronization issues. Correctness could depend on the exact ordering of operations between multiple threads.

15.5 Kernel-Level Debugging

Errors inside xv6 can result in kernel panics, invalid memory accesses, or incorrect process behavior. Debugging required understanding both the code being modified and the surrounding kernel infrastructure.

16. Learning Outcomes

After completing the laboratory exercises, the following practical understanding was developed:

1. **Linux process management**

Ability to inspect and reason about processes using standard Linux tools.

2. **UNIX process APIs**

Practical understanding of `fork`, `exec`, `wait`, and process termination.

3. **Shell implementation**

Understanding of how a command interpreter launches and manages programs.

4. **Inter-process communication**

Experience with mechanisms for communication between isolated processes.

5. **Operating-system internals**

Ability to navigate and modify xv6 kernel code.

6. Virtual memory

Understanding of address spaces, page tables, page permissions, and page faults.

7. Memory-management optimization

Understanding and implementation of Copy-on-Write.

8. Multithreading

Practical experience creating and coordinating POSIX threads.

9. Synchronization

Understanding of locks, reader-writer synchronization, semaphores, fairness, and starvation.

10. Kernel-level threading

Understanding of how threads can be integrated into an operating-system kernel.

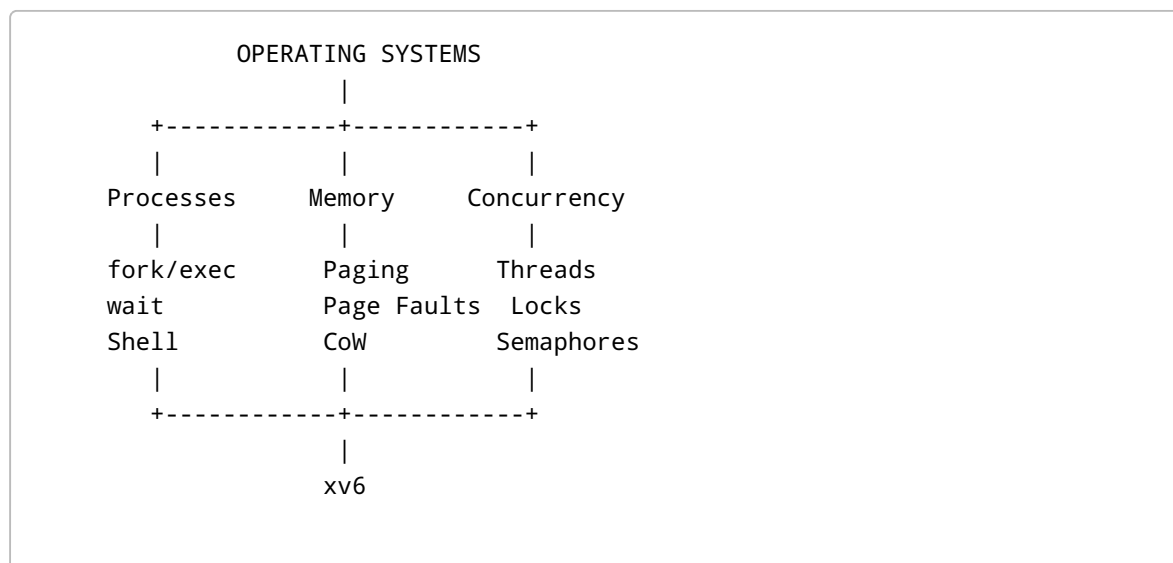
17. Conclusion

The Operating Systems laboratory work provided a progression from using operating-system interfaces to implementing operating-system mechanisms.

The early laboratories established a foundation in Linux and UNIX process management. The shell exercises then demonstrated how these primitives are combined to construct a command interpreter. IPC exercises showed how independent processes can cooperate, while the xv6 laboratories moved into the implementation of operating-system mechanisms themselves.

The memory-management exercises provided practical understanding of virtual address spaces, page tables, page faults, and Copy-on-Write. The concurrency laboratories then introduced POSIX threads and synchronization mechanisms, demonstrating the difficulties associated with shared state and concurrent execution. Finally, the xv6 threading work connected these concepts back to the operating-system kernel by implementing thread-level execution and context management.

Overall, the labs provided hands-on experience across the major components of an operating system:



The most significant outcome of the laboratory work was the transition from treating the operating system as a collection of abstractions to understanding how those abstractions are actually implemented. Working directly with process structures, page tables, context switching, synchronization primitives, and xv6 kernel code provided a systems-level understanding that complements the theoretical study of operating systems.

18. References

1. Mythili Vutukuru, IIT Bombay, **Lectures on Operating Systems** — course lectures, programming exercises, practice problems, and xv6 material.
2. IIT Bombay, **Lab: Introduction to Linux Tools**.
3. IIT Bombay, **Lab: Building a Shell**.
4. IIT Bombay, **Lab: Dynamic Memory Management**.
5. IIT Bombay, **Lab: A Simple Filesystem**.
6. IIT Bombay, **CS333/Operating Systems Laboratory — Copy-on-Write Fork in xv6**.
7. GitHub repository containing the laboratory submissions: `sunny4095/OS-Labs`.

Note: The report intentionally excludes the repository directories explicitly associated with lab quizzes.